

I used ChatGPT and Gemini as my two GenAIs, I used the same copy-pasted prompt for both AIs as follows. I used them through their respective websites directly. The prompt is long so as to include all the instructions, so please zoom in to read it.

i need to ask 2 genais the following so you're first
this will be in C

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

the commands will be as follows:
EMAIL <sender category>, <subject line>, <date>
The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.
EMAIL is followed by space. The rest of the fields are delimited.
Assume <sender category> is one of the five strings listed above.
Assume <subject line> is a string which may contain spaces, but not commas
Assume <date> is in the format: MM-DD-YYYY

NEXT
Next email for the CEO to read. Display the information on the terminal in the following format:
Sender:
Subject:
Date

READ
CEO has read email and dealt with it

COUNT
Display current count of unread files

The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

the following sample test file
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT

should have the following output
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read

The rest of the assignment is on the following pages.

Note that both programs are condensed to solely function names to make them even possible to screenshot

ChatGPT produced the following program:

```

//CHATGPT
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define INITIAL_CAPACITY 10
// Structure to store information about one email
typedef struct {...
} Email;
// Structure for the MaxHeap
typedef struct {...
} MaxHeap;
// Convert sender category into a priority number
int getPriority(char sender[]) {...
}
// Convert MM-DD-YYYY into YYYYMMDD
int convertDate(char date[]) {...
}
// Return 1 if email1 has higher priority than email2
int isHigherPriority(Email email1, Email email2) {...
}
// Swap two emails
void swap(Email *a, Email *b) {...
}
// Initialize the MaxHeap
void initializeHeap(MaxHeap *heap) {...
}
// Increase heap capacity if needed
void resizeHeap(MaxHeap *heap) {...
}
// Move an email upward to restore MaxHeap order
void heapifyUp(MaxHeap *heap, int index) {...
}
// Move an email downward to restore MaxHeap order
void heapifyDown(MaxHeap *heap, int index) {...
}
// Insert an email into the MaxHeap
void insert(MaxHeap *heap, Email email) {...
}
// Return highest-priority email without removing it
Email peek(MaxHeap *heap) {...
}
// Remove highest-priority email
Email removeMax(MaxHeap *heap) {...
}
// Process an EMAIL command
void processEmail(MaxHeap *heap, char line[]) {...
}
// Process a NEXT command
void processNext(MaxHeap *heap) {...
}
// Process a READ command
void processRead(MaxHeap *heap) {...
}
// Process a COUNT command
void processCount(MaxHeap *heap) {...
}
// Main function
int main() {...
}

```

Gemini produced the following program:

```
//GEMINI
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define INITIAL_CAPACITY 10
// Structure to represent an Email
typedef struct { ...
} Email;
// Structure to represent a MaxHeap
typedef struct { ...
} MaxHeap;
// Function to determine sender priority rank
int getRank(const char *category) { ...
}
// Convert MM-DD-YYYY to an integer for chronological comparison (YYYYMMDD)
int getDateVal(const char *dateStr) { ...
}
// Initialize the MaxHeap
MaxHeap* createHeap() { ...
}
// Free the MaxHeap
void freeHeap(MaxHeap *h) { ...
}
// Swap two emails
void swap(Email *a, Email *b) { ...
}
// Compare two emails: returns positive if a > b, negative if a < b, 0 if equal
int compareEmails(Email a, Email b) { ...
}
// Heapify up (Bubble up)
void heapifyUp(MaxHeap *h, int index) { ...
}
// Heapify down (Bubble down)
void heapifyDown(MaxHeap *h, int index) { ...
}
// Insert an email into the MaxHeap
void insertEmail(MaxHeap *h, const char *sender, const char *subject, const char *date) { ...
}
// Peek at the highest priority email
Email* peekMax(MaxHeap *h) { ...
}
// Remove the highest priority email (READ command)
void extractMax(MaxHeap *h) { ...
}
// Trim trailing newline or carriage return characters
void trimString(char *str) { ...
}
int main() { ...
}
```

2E:

- ChatGPT
 - Correctness: ChatGPT's code is functionally correct.
 - Execution time: There are no meaningful execution-time advantages between the two programs.
 - Space complexity: ChatGPT uses a bit more storage because it reserves 256 characters of space for its subject.
 - Maintainability: The code is easy to maintain because it separates each command into its own function that is called from main instead of handling everything within main.
- Gemini
 - Correctness: Gemini's code is functionally correct. However, it doesn't check if memory allocation fails, which could crash the program.
 - Execution time: There are no meaningful execution-time advantages between the two programs.
 - Space complexity: Gemini uses a slightly smaller amount of storage because it only reserves 100 characters of space for its subject.
 - Maintainability: Gemini's code handles identifying commands, processing emails, and handling output in its main function, which could make it harder to maintain. However, it keeps its heap functions as separate easily named functions.

2F: I would choose Gemini's program as my basis because it uses less memory per email. The maintainability is a bit of an issue, but I think it can be easily fixed.

2G: I would improve Gemini's code by adding better string handling and memory allocation checks so the program can't throw up errors. I would also separate the EMAIL input processing into its own function and add comments to make it easier to maintain.